

Table of Contents

1. Introduction.....	1
2. Implementation Overview.....	1
3. Experimental Setup.....	1
4. Data Structure used.....	2
5. C++ Code Skeleton (LPA*).....	2
6. C++ Code Skeleton (D* Lite).....	4
7. Results.....	6
8. Comparison of LPA* and D* Lite Algorithms.....	6
9. Conclusion.....	7
10. Snapshots.....	7

Introduction

This report presents the implementation and evaluation of two path-planning algorithms: Lifelong Planning A* (LPA*) and D* Lite in C++ programming language in an 8-connected grid world. Both algorithms were applied to different grid-based pathfinding problems to compare their performance under dynamic conditions. The aim is to evaluate their efficiency, adaptability, and scalability across different maps and heuristics. This program can execute algorithms using different parameters to record and analyze their performance. Heuristic functions: Euclidean distance and Chebyshev distance. A cross-platform graphics start-up program that is provided is used to plot the optimal path generated and view the details of the variables involved in the computations.

Implementation Overview

The system is implemented in C++ and integrates with a GridWorld environment. The algorithms were executed in two phases: initial planning and replanning after a environment change. Both Chebyshev and Euclidean heuristics were implemented and tested. The experimental setup automatically loads grid maps, runs the algorithms, and records performance metrics.

Key performance metrics collected for both algorithms include:

- Path length
- Number of expansions
- Maximum queue size
- Vertex accesses
- Runtime

Experimental Setup

Experiments were conducted on six grid maps of varying complexity:

- grid_dstar_journal.map
- grid_lpa_journal.map
- grid_lpa_journal_big.map
- grid_spiral.map
- grid_trap.map
- grid_big.map

Each algorithm was tested using both Chebyshev and Euclidean heuristics. For LPA*, results are reported separately for initial and final planning phases. For D* Lite, results are reported for both the initial run and after a simulated obstacle was introduced.

Data Structure Used

Lifelong Planning A* (LPA*)

- LpaStarCell: A custom struct representing each grid cell, containing:
 - g and rhs.
 - h - heuristic to goal.
 - key[2] - the priority key used in the open list.
 - trace pointer - to reconstruct paths.
- Open List (U): Implemented as a map storing states with their keys. The algorithm uses calcKey() to update entries, and insertOrUpdate() to maintain the open list.
- Neighbor Access: getNeighbors() collects up to four neighbors (N, S, E, W) for each cell.
- Supporting Variables: Counters for state expansions, vertex accesses, path length, queue size, and runtime.

D* Lite

- vertex: g, rhs, and a trace pointer.
- NodeKey: A struct storing a pair (k1, k2) along with a pointer to a vertex. This represents a priority in the open list.
- Priority Queue (U): Implemented as a std::set of NodeKey elements with a custom comparator (NodeKeyCmp). This keeps the queue always sorted by priority.
- InQueue Map: A hash map (unordered_map<vertex*, pair<double,double>>) to quickly check and update the current key of each vertex.
- Neighbors: getNeighbors() considers up to 8 neighbors.
- Supporting Variables: D* Lite also tracks expansions, vertex accesses, max queue size, path length, and runtime.

C++ Code Skeleton

Lifelong Planning A* (LPA*)

```
class LpaStar {  
private:  
    struct Cell {
```

```

    int x, y;
    double g, rhs, h;
    double key[2];
    bool obstacle;
    Cell* trace;
};

std::vector<std::vector<Cell>> maze;
std::map<Cell*, std::pair<double,double>> U;

Cell* start;
Cell* goal;

public:
    LpaStar(int rows, int cols, int heuristic);

    void initialise(int sx, int sy, int gx, int gy) {
        // set g, rhs =  $\infty$ , rhs(start)=0
        // insert start into open list
    }

    double heuristicDist(const Cell& a, const Cell& b) {
        // Euclidean or Chebyshev
    }

    std::pair<double,double> calcKey(Cell* u) {
        // return (min(g, rhs) + h, min(g, rhs))
    }

    void insertOrUpdate(Cell* u) {
        // maintain in open list
    }

    std::vector<Cell*> getNeighbors(Cell* u) {
        // return N, S, E, W neighbors
    }

    void updateVertex(Cell* u) {

```

```

    // rhs(u) = min over neighbors (g + cost)
    // update in queue
}
void computeShortestPath() {
    while (!U.empty() && (condition not satisfied)) {
        // pop smallest key
        // if key changed, reinsert
        // else if g > rhs, set g = rhs and update neighbors
        // else set g =  $\infty$  and update
    }
}
void extractPath() {
    // follow trace from goal to start
}
};

```

D* Lite:

```

class DStarLite {
private:
    struct Vertex {
        int row, col;
        double g, rhs;
        Vertex* trace;
    };
    struct NodeKey {
        Vertex* v;
        double k1, k2;
    };
    struct NodeKeyCmp {
        bool operator()(const NodeKey& a, const NodeKey& b) const {
            // sort by (k1, k2)
        }
    };
};

```

```

std::set<NodeKey, NodeKeyCmp> U;
std::unordered_map<Vertex*, std::pair<double,double>> inQueue;

Vertex* s_start;
Vertex* s_goal;
double km;

public:
    DStarLite(int rows, int cols, int heuristic);

    void initialise(int sx, int sy, int gx, int gy) {
        // g, rhs =  $\infty$  for all
        // rhs(goal) = 0
        // insert goal into queue
    }

    double heuristicBetween(Vertex* a, Vertex* b) {
        // Euclidean or Chebyshev
    }

    std::pair<double,double> calculateKey(Vertex* u) {
        // return (min(g, rhs) + heuristic + km, min(g, rhs))
    }

    void insertOrUpdate(Vertex* u) {
        // maintain in queue
    }

    std::vector<Vertex*> getNeighbors(Vertex* u) {
        // return up to 8 neighbors (including diagonals)
    }

    void updateVertex(Vertex* u) {
        // rhs(u) = min over neighbors (g + cost)
        // update queue
    }

    void computeShortestPath() {
        while (!U.empty() && (goal condition not met)) {

```

```

        // process top of queue
        // update g/rhs and predecessors
    }
}
void extractPath() {
    // follow trace from start to goal
}
void finalPlanning() {
    // simulate blocked cell
    // re-run computeShortestPath()
}
};

```

Results

The following observations were made from the experiments:

- LPA* successfully planned paths and handled replanning, but for some maps (e.g., grid_big.map), no valid path was found, which is expected due to the map configuration.
- D* Lite handled dynamic changes efficiently and updated paths after obstacle insertion. Its replanning was often faster than LPA* since it reused previous computations more effectively.
- Path lengths produced by both heuristics were consistent, confirming correctness. The difference between heuristics was mainly in expansions and runtime.
- For large maps such as grid_spiral.map and grid_big.map, runtime and expansions increased significantly, showing the scalability challenges.

Comparison of LPA* and D* Lite Algorithms

From the experiments, the following comparison can be made:

LPA*:

- Performs well for initial and replanning phases.
- Tracks maximum queue size and vertex accesses.
- May fail to find a path if the map configuration blocks the start-goal connection.

D* Lite:

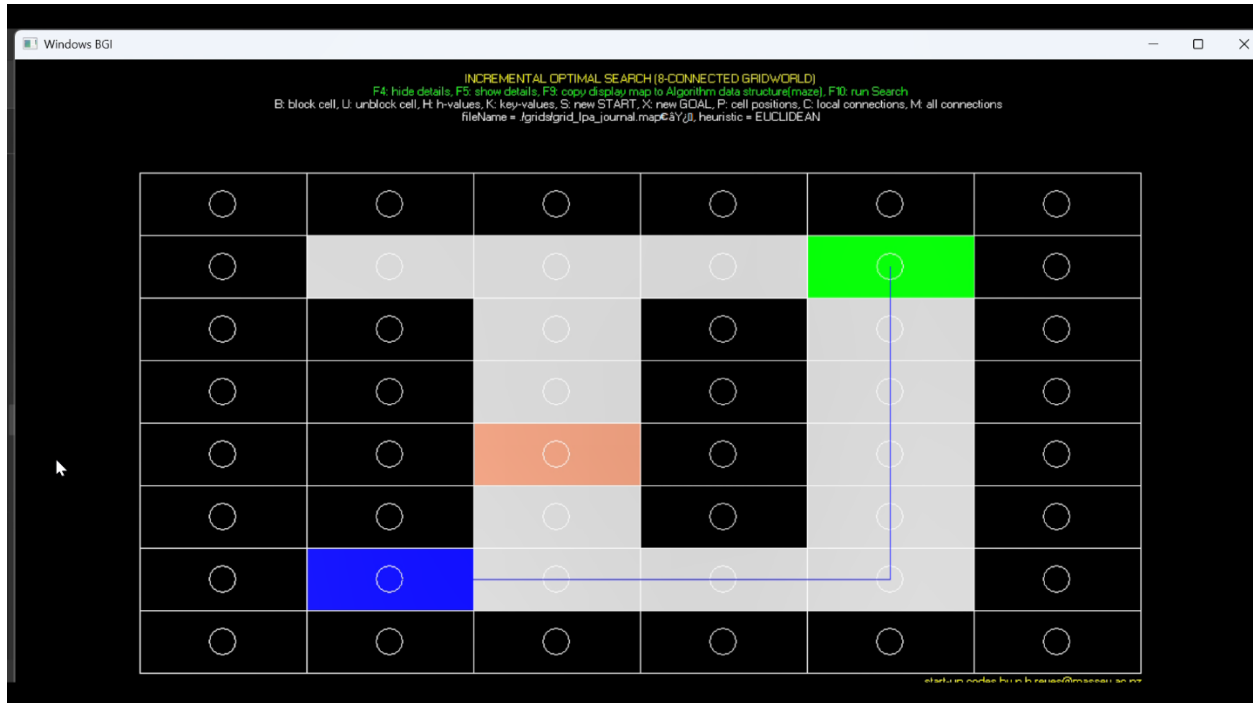
- More efficient in handling dynamic changes with fewer expansions.
- Tracks vertex accesses directly and adjusts paths incrementally.
- Provides consistent path quality while often requiring fewer updates.

Conclusion

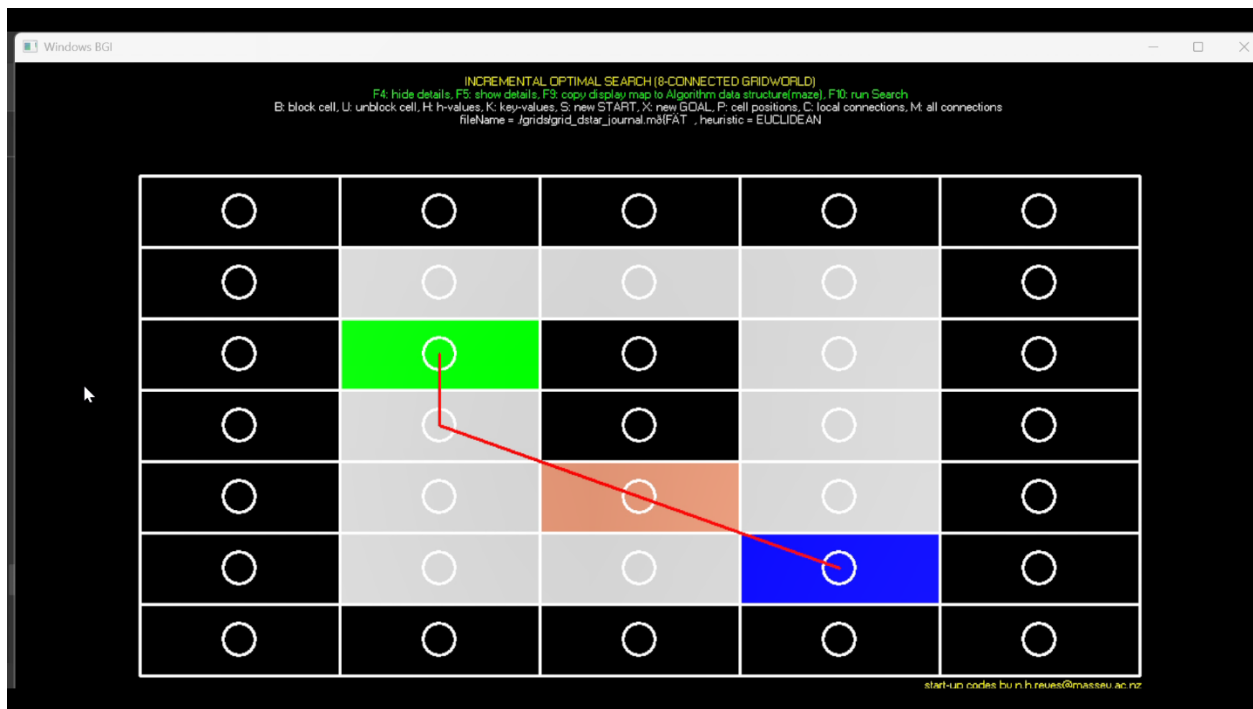
This project successfully implemented and tested two replanning algorithms, LPA* and D* Lite, on a variety of grid-based maps with Chebyshev and Euclidean heuristics. Both algorithms produced correct and consistent path solutions, with D* Lite showing advantages in dynamic replanning scenarios due to their incremental nature. The experiments demonstrate the trade-offs between initial planning efficiency and replanning speed. Future improvements could focus on optimizing heuristics for specific environments and extending the framework to handle larger or more complex grid maps.

Snapshots

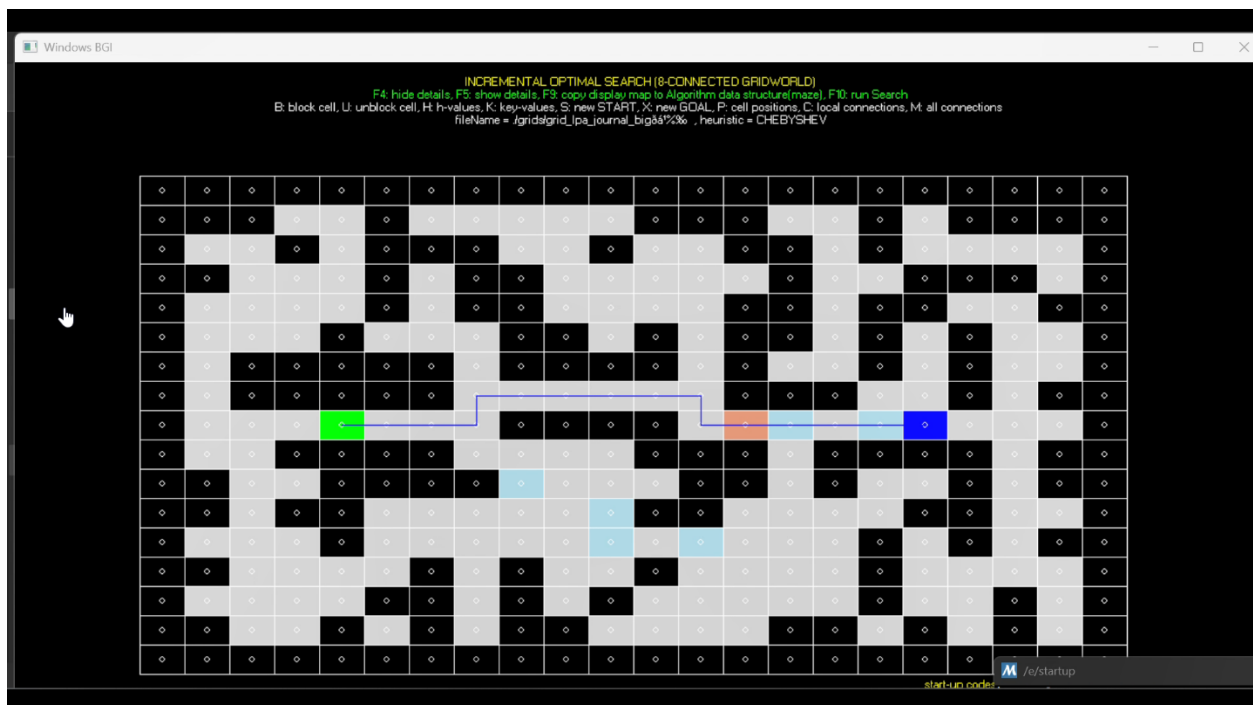
Algorithm: LPA* Map: grid_lpa_journal Heuristic: Euclidean



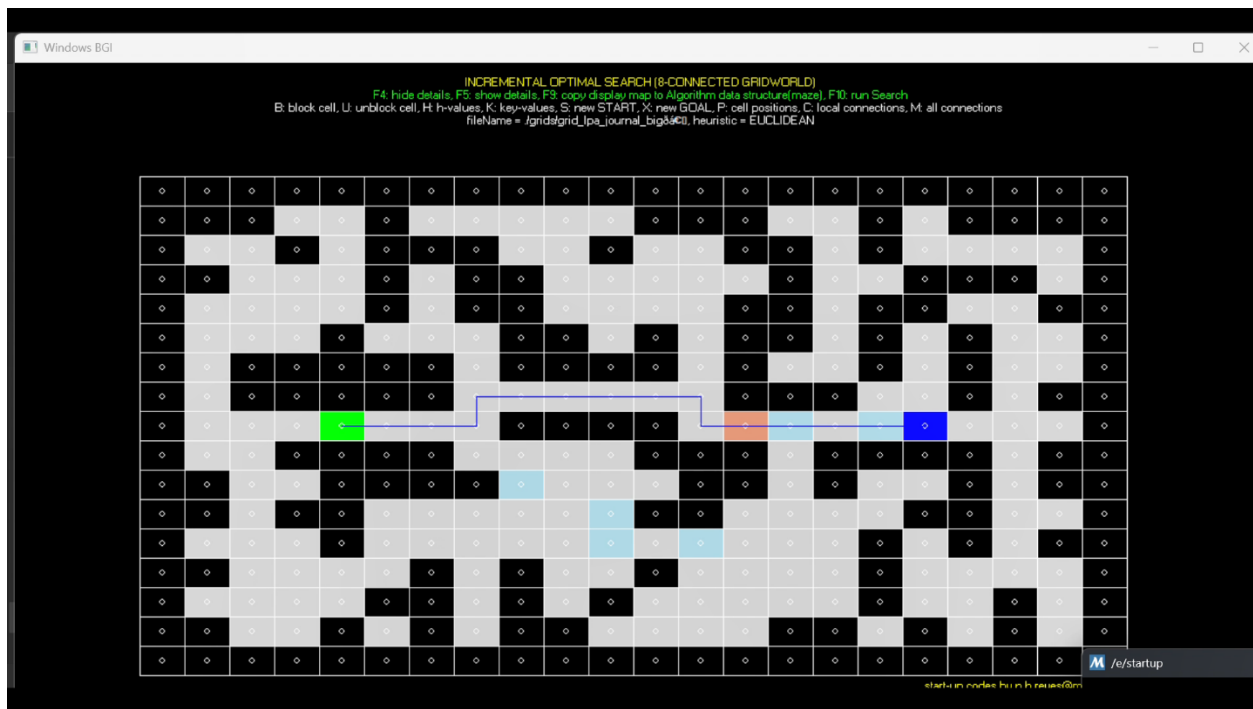
Algorithm: D* Lite Map: grid_dstar_journal Heuristic: Euclidean



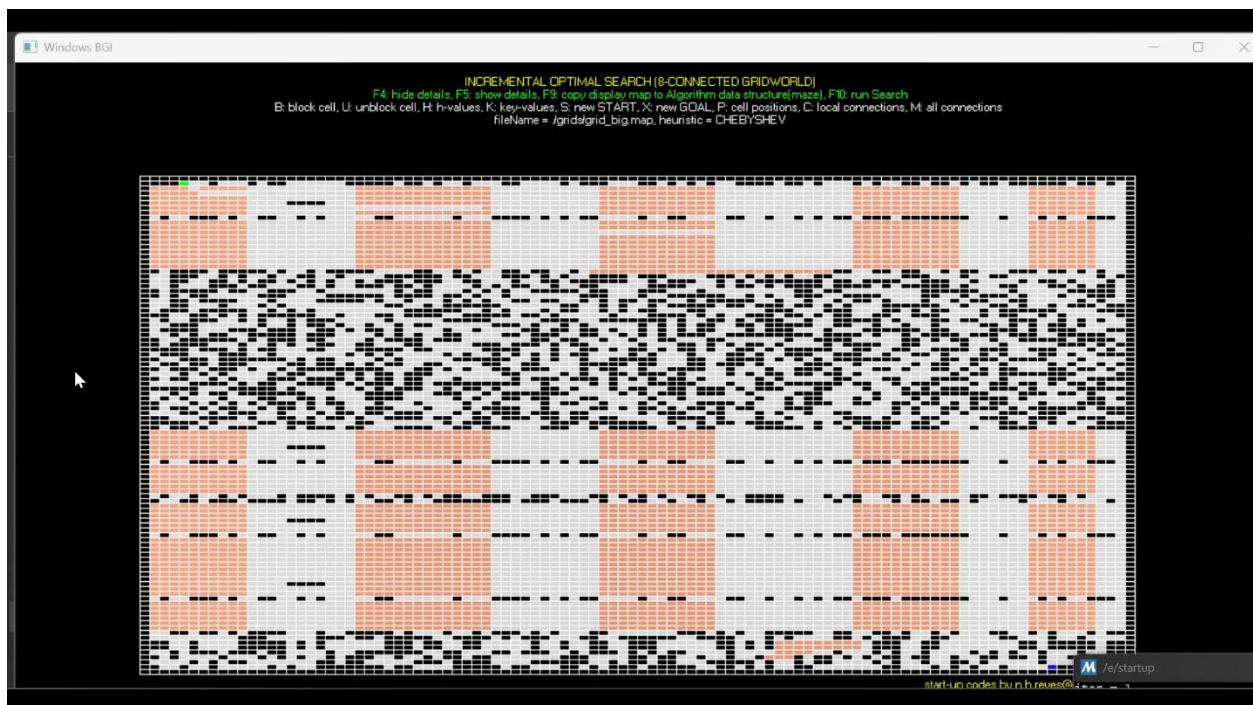
Algorithm: LPA* Map: grid_lpa_journal_big Heuristic: Chebyshev



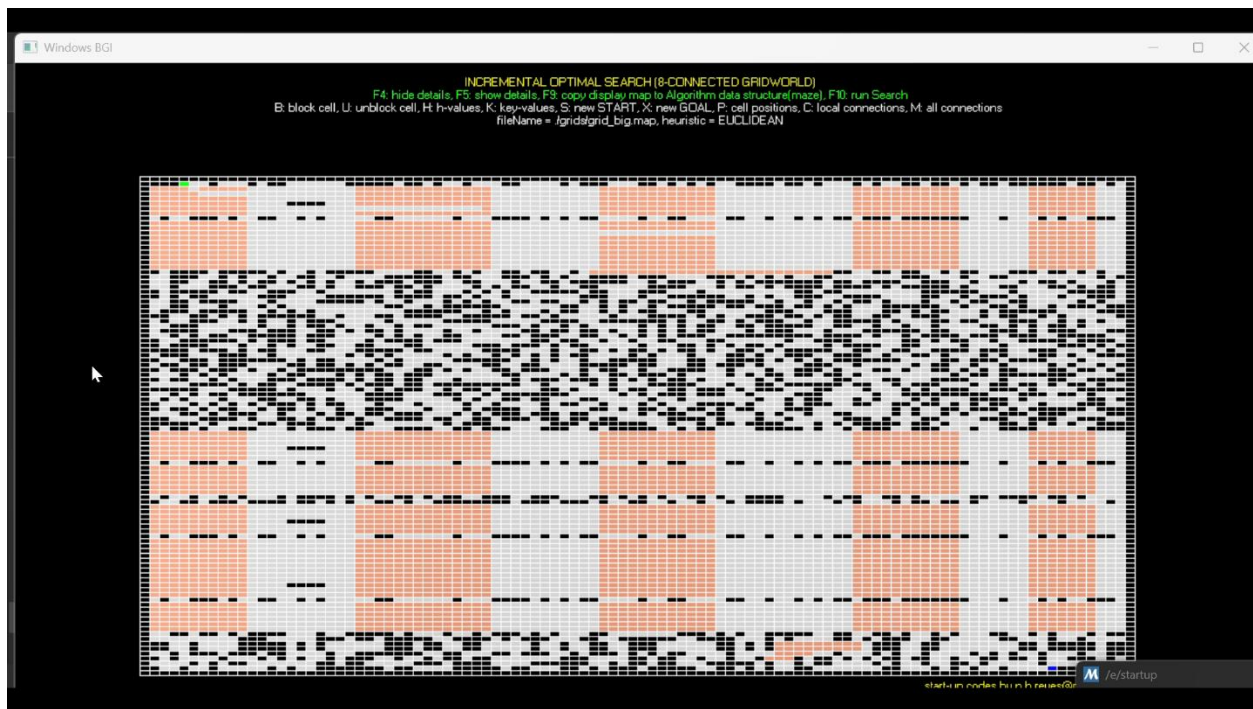
Algorithm: LPA* Map: grid_lpa_journal_big Heuristic: Euclidean



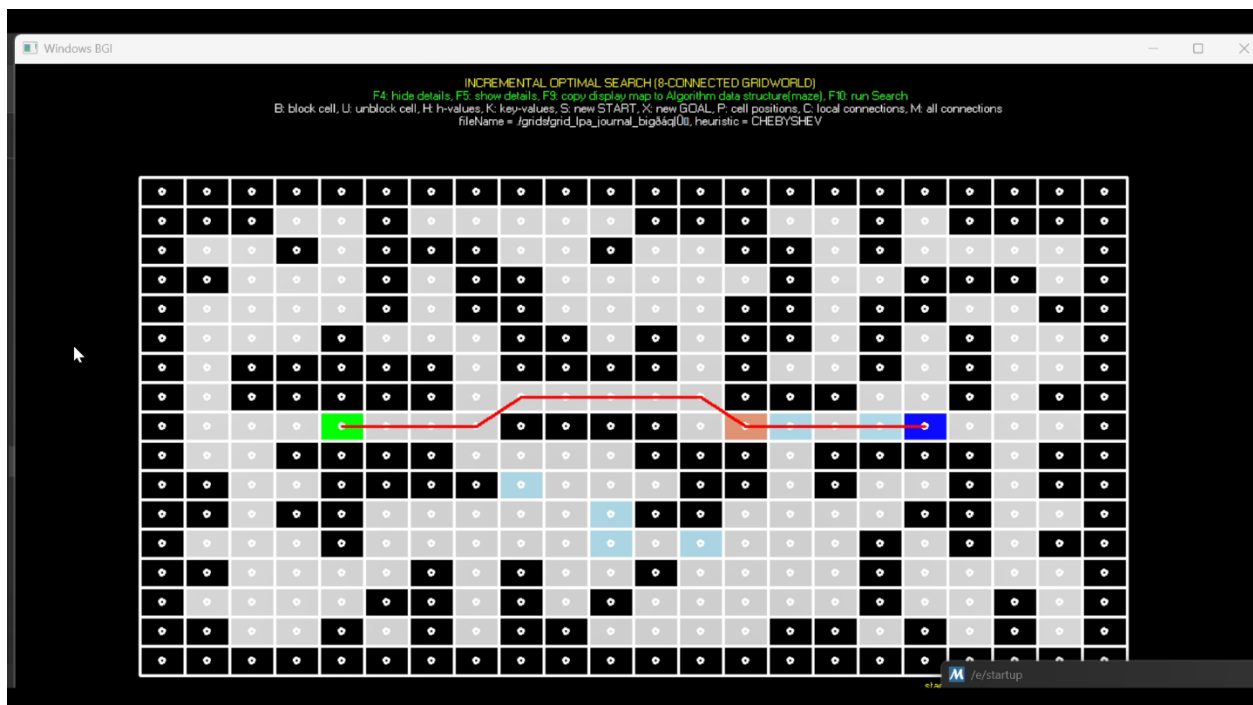
Algorithm: LPA* Map: grid_big Heuristic: Chebyshev



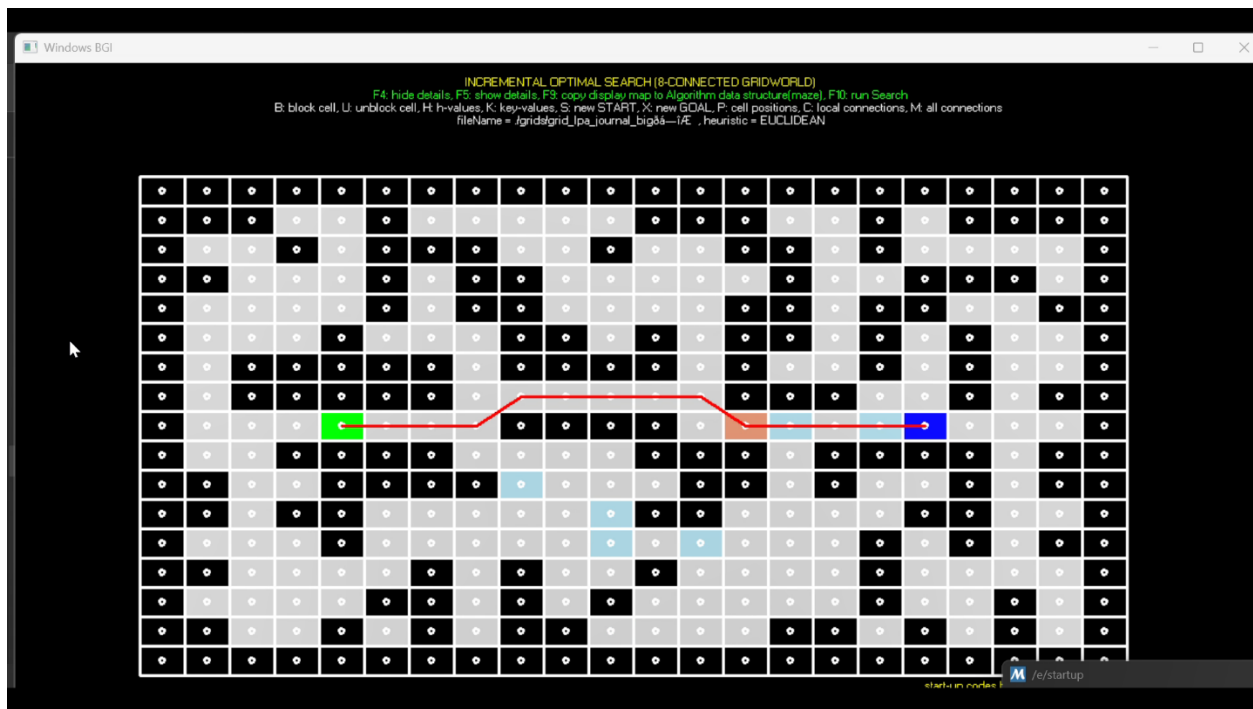
Algorithm: LPA* Map: grid_big Heuristic: Euclidean



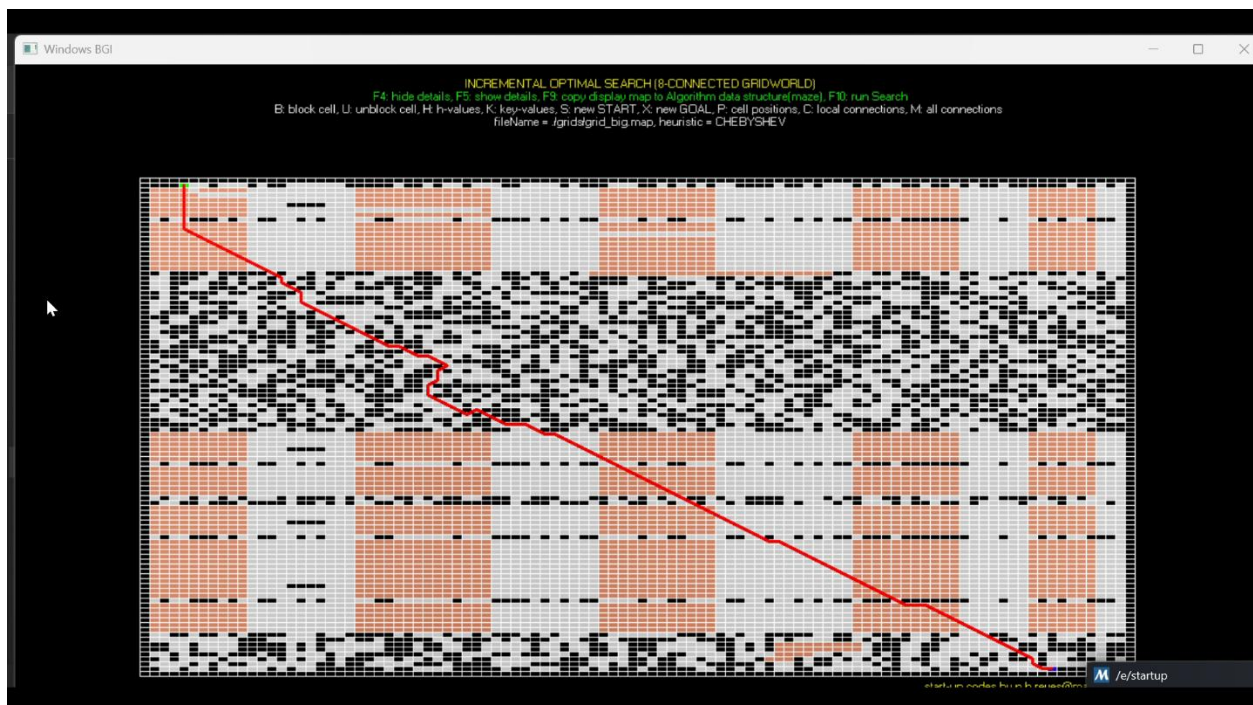
Algorithm: D* Lite Map: grid_lpa_journal_big Heuristic: Chebyshev



Algorithm: D* Lite Map: grid_lpa_journal_big Heuristic: Euclidean



Algorithm: D* Lite Map: grid_big Heuristic: Chebyshev



Algorithm: D* Lite Map: grid_big Heuristic: Euclidean

